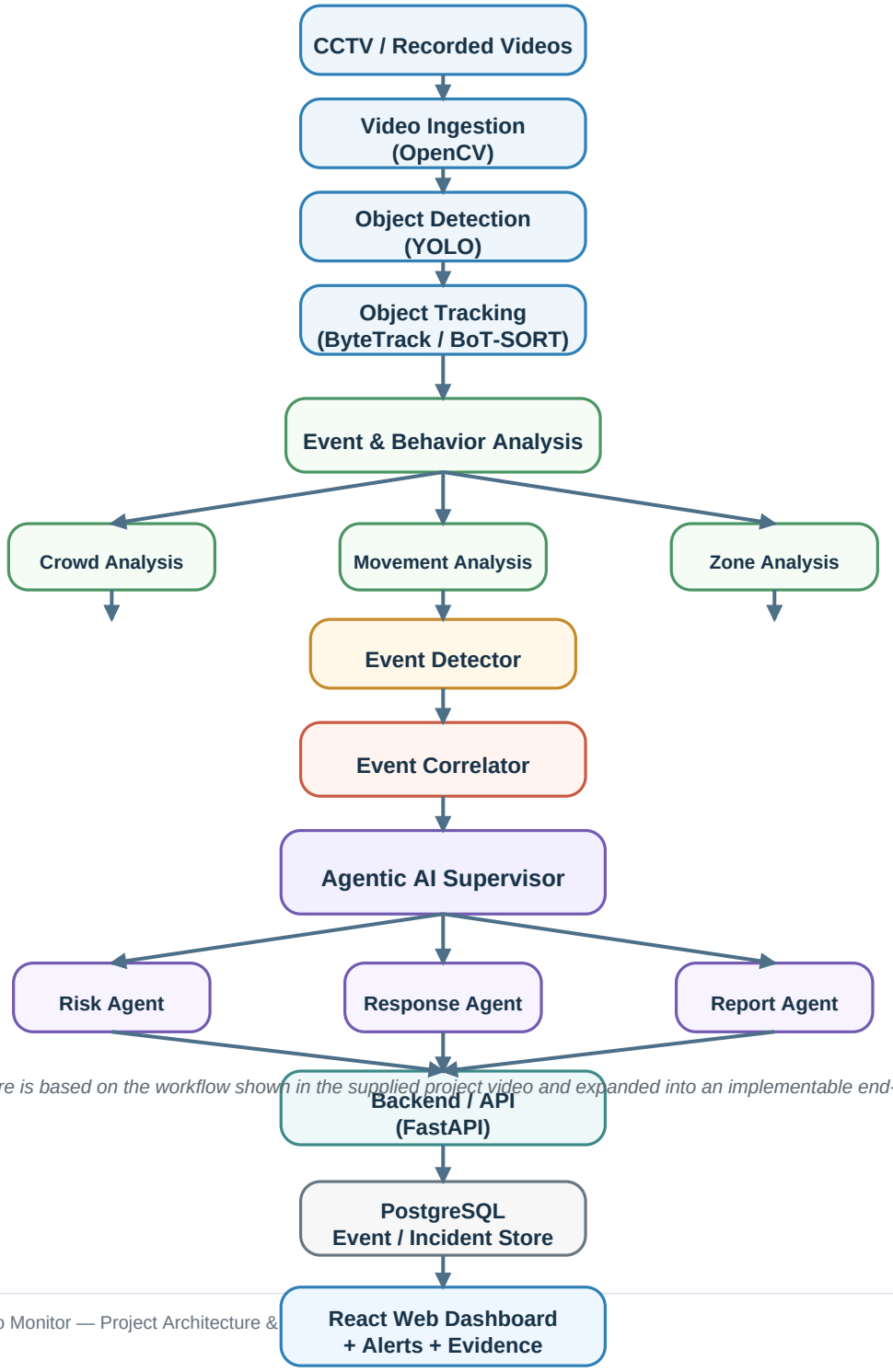
AI PUBLIC SAFETY VIDEO MONITOR

Computer Vision + Agentic AI for Real-Time Public Safety Monitoring

---

Complete Project Architecture, Workflow, Technology Stack, Real-Time Example, Implementation Guide and Evaluation Plan



The architecture is based on the workflow shown in the supplied project video and expanded into an implementable end-to-end system.

Document Contents

1. Project vision and problem definition
2. What the final system does
3. Complete end-to-end architecture
4. Detailed computer vision pipeline
5. Crowd, movement and zone analysis
6. Event detection and event correlation
7. Agentic AI architecture
8. Risk, response and report agents
9. Backend, database and real-time communication
10. Web dashboard and user experience
11. Real-time incident walkthrough
12. False-alert reduction and explainability
13. Multi-camera extension
14. Technology stack and recommended tools
15. Data structures and module interfaces
16. Functional and non-functional requirements
17. MSE / first milestone definition of done
18. Final project roadmap
19. Evaluation and research methodology
20. Security, privacy and responsible-use considerations
21. Final system checklist

Core design principle

Build one integrated pipeline. Computer Vision should produce reliable structured events; the Agentic AI layer should reason over those events; the backend should store and distribute the results; and the dashboard should present actionable information.

1. Project Vision and Problem Definition

Public spaces such as college campuses, gates, corridors, stations, event venues and building entrances can experience crowd buildup, unusual movement, falls, restricted-area intrusion or other abnormal situations. Human operators may not notice every change in time, especially when several camera feeds must be watched simultaneously.

The proposed system continuously analyzes camera video using Computer Vision and converts raw visual activity into structured safety events. An Agentic AI layer then combines those events, estimates the seriousness of an incident and decides what response workflow should be triggered.

Important: The system is intended as a decision-support and monitoring system, not as an autonomous law-enforcement system. Human verification should remain part of the operational workflow for high-impact decisions.

Project goal

Build a real-time AI safety monitoring platform that can detect selected safety-relevant patterns, correlate multiple signals, assess risk, preserve evidence, and present a clear incident summary through a web dashboard.

Recommended core capabilities

- **Person detection** — identify people in video frames.
- **Multi-object tracking** — maintain person IDs across frames.
- **Crowd counting and density** — estimate the number and concentration of people in an area.
- **Restricted-zone intrusion** — detect entry into predefined zones.
- **Loitering** — identify a person remaining in a region longer than a configured threshold.
- **Abnormal movement** — detect sudden running, direction changes or movement against the dominant crowd direction.
- **Fall detection** — identify a possible fall and verify that the person remains down for a short observation period.
- **Incident correlation** — combine multiple events into one incident.
- **Agentic risk assessment** — reason over structured events and assign severity.
- **Response and reporting** — save evidence, create alerts and generate a human-readable incident report.

2. What the Final System Does

At a high level, the final product follows this sequence:

Camera → Video Ingestion → Detection → Tracking → Behavior Analysis → Event Detection → Event Correlation → Agentic AI → Risk → Response → Evidence/Report → Backend → Dashboard

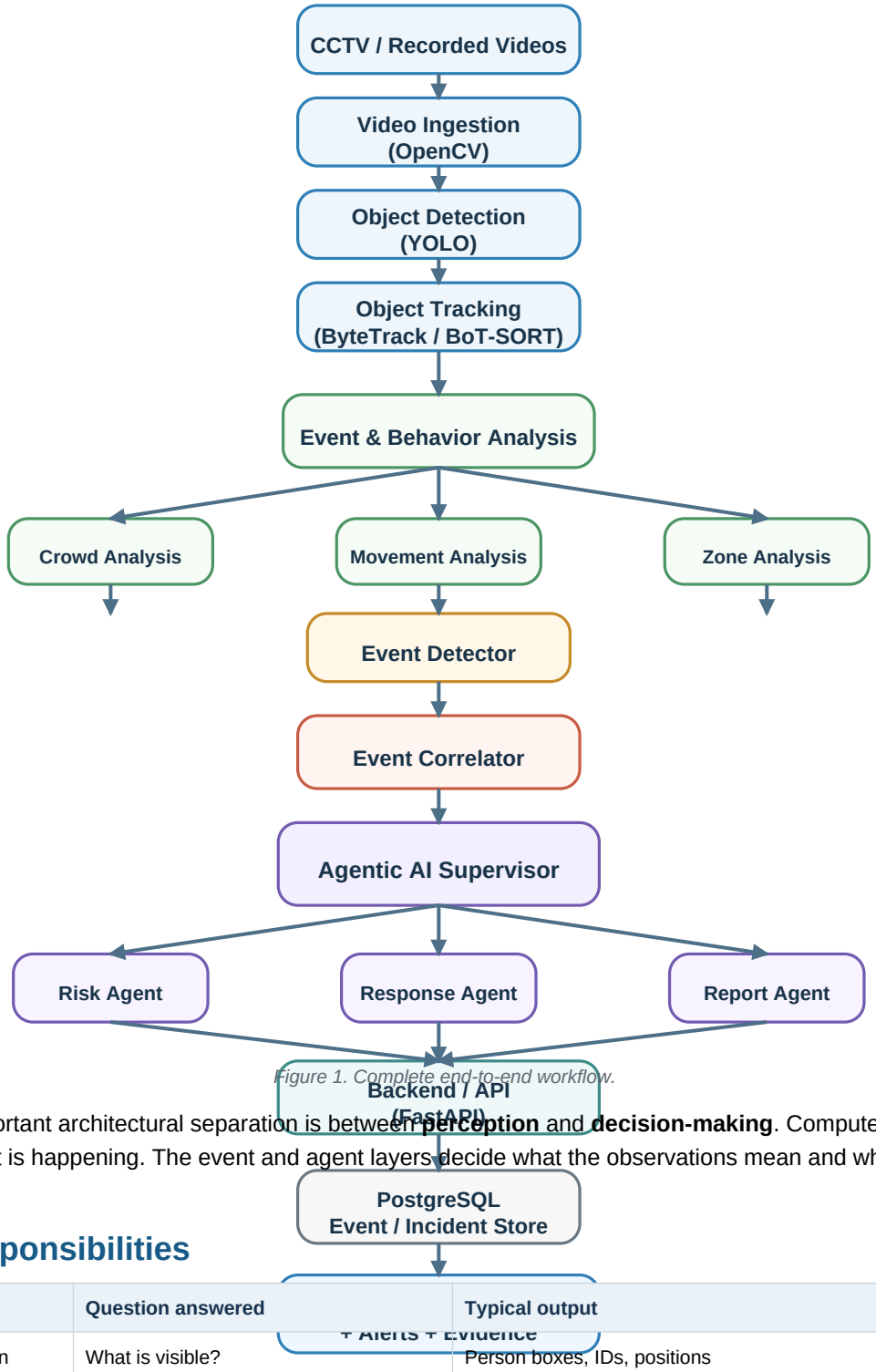


Figure 1. Complete end-to-end workflow.

The most important architectural separation is between **perception** and **decision-making**. Computer Vision observes what is happening. The event and agent layers decide what the observations mean and what should happen next.

Layer responsibilities

Layer	Question answered	Typical output
Computer Vision	What is visible?	Person boxes, IDs, positions
Behavior Analysis	What is changing?	Density, speed, direction, duration
Event Detection	What happened?	Crowd buildup, fall, intrusion
Event Correlation	Are events related?	One combined incident

Layer	Question answered	Typical output
Agentic AI	How serious is it and what next?	Risk, action plan, explanation
Backend	Where is information stored/shared?	API responses, database records
Dashboard	What should the human see?	Live status, alerts, evidence

3. Architecture Reference from the Supplied Video

The supplied video presents the central architecture as a vertical flow beginning with CCTV/videos, followed by video ingestion, YOLO detection, tracking, event processing, an Agentic AI Supervisor, specialized agents, FastAPI backend and a web dashboard. The following screenshots are included as visual references; the implementation diagrams in this report use clearer labels and the corrected term **Event Correlator**.

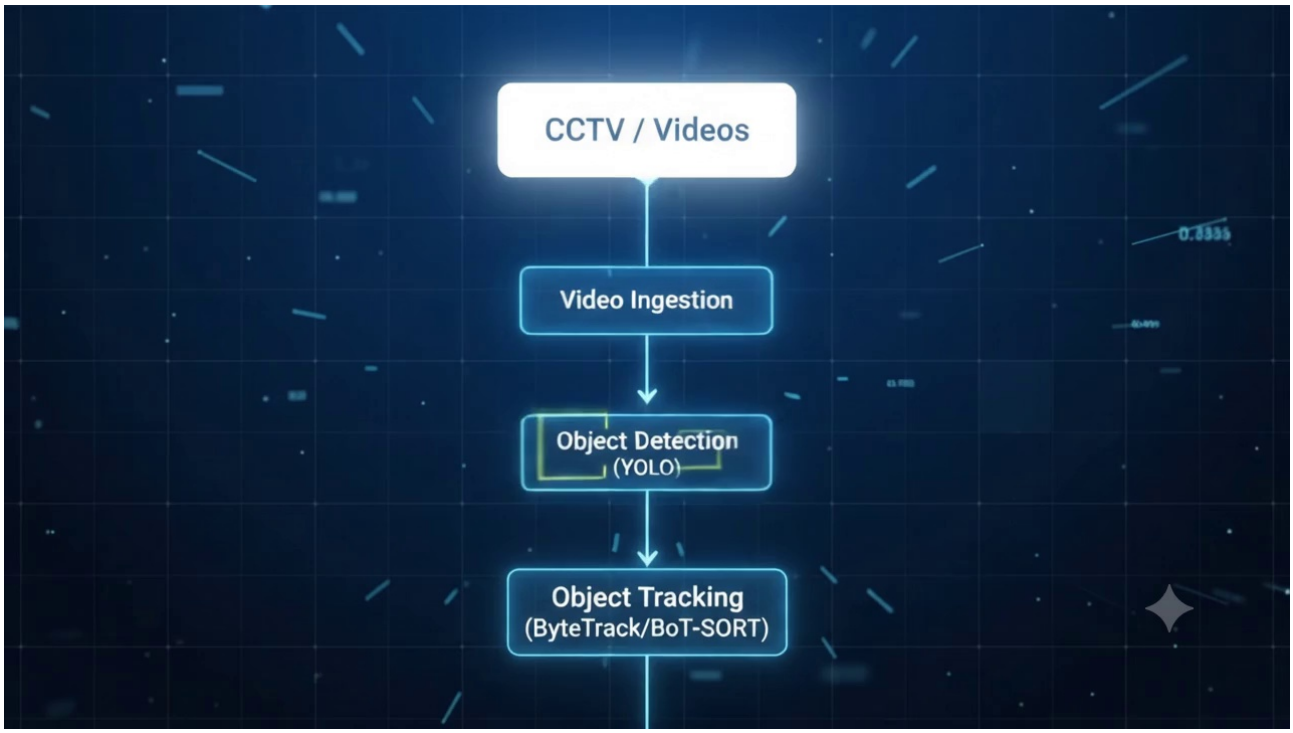


Figure 2. Reference frame: input, ingestion, YOLO detection and tracking.

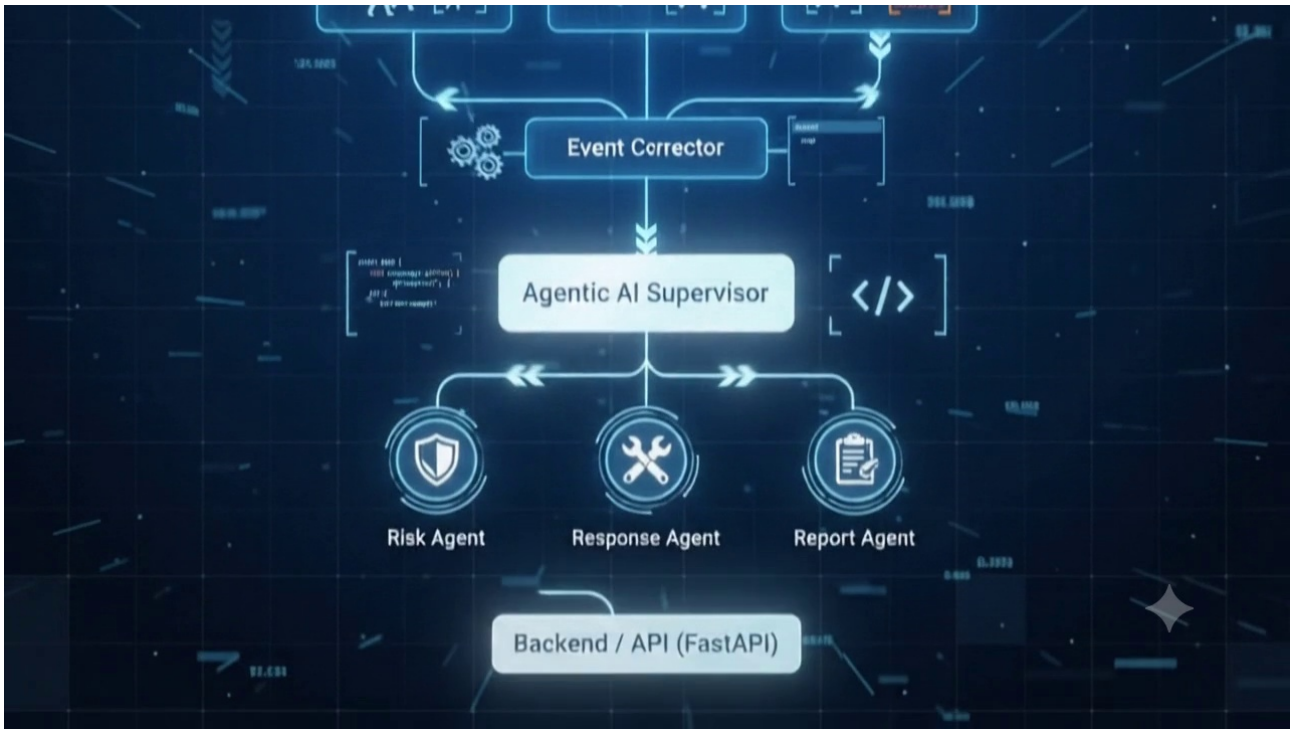


Figure 3. Reference frame: Event Correlator, Agentic AI Supervisor, specialized agents and FastAPI.



Figure 4. Reference frame: React web dashboard and risk/incident visualization.

4. Detailed Computer Vision Pipeline

The Computer Vision layer is the foundation. It converts video frames into structured information that downstream modules can understand.

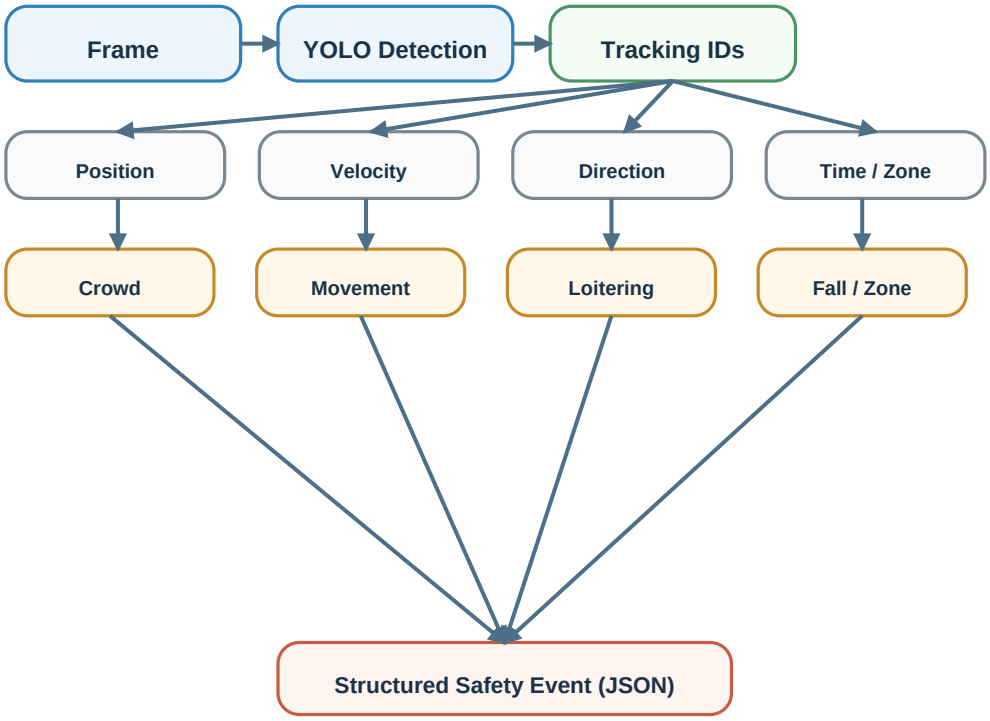


Figure 5. Computer Vision branch from frame processing to structured safety events.

4.1 Video ingestion

Use OpenCV to read frames from recorded videos, webcam input or, later, an RTSP stream. The first implementation should use recorded videos because they are deterministic and easier to debug.

4.2 Object detection — YOLO

YOLO detects objects in each frame and returns a class label, confidence score and bounding box. For the initial system, person detection is the most important capability.

```
camera_id: CAM01
frame_id: 1842
class: person
confidence: 0.94
bbox: [x1, y1, x2, y2]
```

4.3 Object tracking — ByteTrack or BoT-SORT

Tracking associates detections across consecutive frames. A tracked person receives an ID such as Person 17, allowing the system to estimate trajectory, speed, direction, time spent in a region and movement patterns.

4.4 Derived signals

- **Position:** center point or foot point of a bounding box.
- **Velocity:** change in position over time.
- **Direction:** movement vector or dominant direction.
- **Duration:** how long a person remains in a scene or zone.
- **Zone membership:** whether the person is inside a configured polygon.
- **Density:** people per selected region or normalized occupancy estimate.

The CV layer should emit structured events rather than sending raw model outputs directly to the user interface.

5. Crowd, Movement and Zone Analysis

These analysis modules sit on top of detection and tracking. They transform individual person trajectories into safety-relevant patterns.

5.1 Crowd analysis

- Count people in a selected region of interest.
- Estimate occupancy or density.
- Measure the rate at which the crowd grows or shrinks.
- Identify rapid crowd buildup.
- Optionally estimate the dominant movement direction of the crowd.

A useful crowd signal is not just the current count. The **rate of change** can be more informative. For example, 50 people may be normal at a large event but concerning if a small gate changes from 15 to 50 people within a short interval.

5.2 Movement analysis

- Detect unusually high movement speed.
- Detect sudden acceleration or direction change.
- Compare individual direction with the dominant crowd direction.
- Detect sudden dispersion or convergence.
- Use short temporal windows rather than making a decision from one frame.

5.3 Loitering

If a tracked person remains inside a defined region for more than a configurable duration while moving very little, create a possible loitering event. Thresholds should be configurable because a waiting area and a restricted corridor have different meanings.

5.4 Restricted-zone intrusion

Define a polygonal region on the camera view. If a tracked person's reference point enters the polygon, create an intrusion event. A small temporal confirmation window can reduce alerts caused by detection noise.

5.5 Fall detection

A robust first version should use a temporal rule rather than a single-frame classification: detect a rapid posture/position change, observe a low or horizontal body position, and verify that the person remains down for several seconds before raising a high-priority event.

The objective is to distinguish a possible fall from harmless actions such as sitting, crouching or bending.

6. Event Detection and Event Correlation

The Event Detector converts measurements into named events. The Event Correlator then groups related events into an incident.

6.1 Event examples

Event	Example trigger	Important fields
crowd_buildup	Density and count rising rapidly	people_count, density, growth_rate
abnormal_movement	Speed/direction differs strongly from baseline	person_id, speed, direction
restricted_intrusion	Person enters restricted polygon	person_id, zone_id
loitering	Person stays in region beyond threshold	person_id, duration, zone_id
possible_fall	Rapid change + person remains down	person_id, duration, confidence

6.2 Structured event example

```
{
  "camera_id": "CAM01",
  "timestamp": "2026-09-07T14:32:15",
  "event_type": "possible_fall",
  "person_id": 32,
  "confidence": 0.87,
  "duration_down_sec": 7
}
```

6.3 Event correlation

Correlation looks at events that are close in time, location and context. The goal is to avoid treating every signal as an independent alarm.

Crowd buildup + abnormal movement + possible fall + sudden crowd direction change → one possible crowd emergency incident.

Correlation rules can initially be deterministic and transparent. Later, the Agentic AI layer can use the correlated event bundle as input for deeper reasoning.

6.4 Incident object

```
incident_id: INC-104
camera_id: CAM01
start_time: 14:32:01
incident_type: possible_crowd_emergency
severity: CRITICAL
risk_score: 91
related_events: [crowd_buildup, abnormal_movement, possible_fall]
```

7. Agentic AI Architecture

The Agentic AI layer is the project's main intelligence layer. It should not replace the Computer Vision models. Instead, it reasons over structured events produced by the CV and event-processing layers.

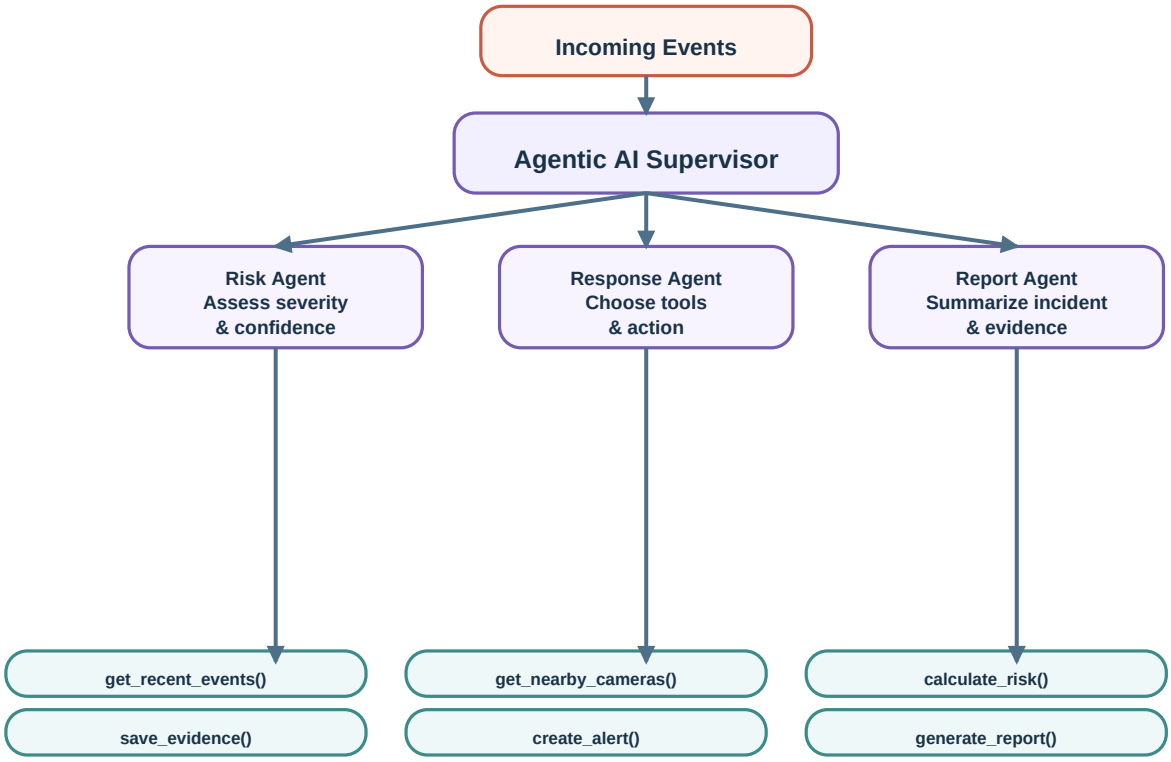


Figure 6. Agentic architecture: a supervisor coordinates specialized agents and tool calls.

7.1 What makes this agentic?

- The system receives an event context.
- A supervisor decides which reasoning step is needed.
- Specialized agents have distinct responsibilities.
- Agents can call tools such as recent-event lookup, camera lookup, risk calculation and evidence saving.
- The result can trigger an operational workflow rather than only generating text.
- The system keeps a structured incident state so later steps can use earlier results.

7.2 Supervisor role

The supervisor receives a bundle of recent events and decides which agent or tool should be used. It should be constrained by structured schemas and explicit allowed actions.

A practical design is one Supervisor with three specialized agents: Risk Agent, Response Agent and Report Agent. Avoid creating many agents just for the sake of calling the system multi-agent.

7.3 Agent tools

```
get_camera_status()
get_recent_events()
get_nearby_cameras()
get_crowd_density()
get_incident_history()
calculate_risk()
save_evidence()
create_alert()
generate_report()
```

8. Risk, Response and Report Agents

8.1 Risk Agent

The Risk Agent estimates severity from multiple signals. A transparent first implementation can combine normalized factors with configurable weights, then optionally allow an LLM-based reasoning step to explain the result.

```
Risk = 0.30 × crowd_score + 0.25 × movement_score + 0.25 × event_score + 0.20 × duration_score
```

Suggested severity bands:

Score	Severity	Meaning
0–30	LOW	Normal or minor observation
31–60	MEDIUM	Needs attention / monitoring
61–80	HIGH	Likely safety concern
81–100	CRITICAL	Immediate human attention recommended

8.2 Response Agent

The Response Agent maps severity and incident type to approved actions. Example actions include saving an evidence clip, creating an alert, checking nearby cameras, escalating to a human operator and starting enhanced monitoring.

The agent should not be allowed to take unrestricted real-world actions. Tool permissions should be explicit, auditable and reversible where possible.

8.3 Report Agent

The Report Agent turns the structured incident state into a concise incident summary containing time, camera/location, detected events, risk score, evidence references, reasoning and recommended next steps.

```
INCIDENT REPORT
Time: 14:32
Camera: Gate 2
Events: Crowd buildup; abnormal movement; possible fall
Risk: 91/100
Severity: CRITICAL
Action: Evidence saved; security notification recommended
```

9. Backend, Database and Real-Time Communication

FastAPI acts as the service layer between Computer Vision, Agentic AI, database storage and the dashboard.

9.1 Backend responsibilities

- Receive structured CV events.
- Store and retrieve events and incidents.
- Expose camera and incident APIs.
- Manage evidence metadata.
- Provide real-time updates through WebSockets.
- Keep authentication and authorization boundaries.
- Log system actions for debugging and audit.

9.2 Suggested API surface

Method	Endpoint	Purpose
GET	/cameras	List camera status and metadata
GET	/events	Retrieve recent or filtered events
POST	/events	Receive a structured CV event
GET	/incidents	List incidents
GET	/incidents/{id}	Get one incident with related events
POST	/alerts	Create an alert
GET	/evidence/{id}	Retrieve evidence metadata
WS	/ws/events	Push real-time event updates

9.3 PostgreSQL data model

Table	Important fields
cameras	id, name, location, stream_url, status
events	id, camera_id, event_type, timestamp, confidence, metadata
incidents	id, camera_id, start_time, end_time, incident_type, severity, risk_score, explanation, status
alerts	id, incident_id, alert_type, timestamp, status
evidence	id, incident_id, file_path, timestamp, type

9.4 Real-time flow

CV event → POST /events → event stored → correlation/agent workflow → incident updated → WebSocket message → React dashboard updates.

10. Web Dashboard and User Experience

The dashboard should make the system understandable without exposing implementation complexity. A user should be able to answer five questions quickly: Which camera is active? What is happening? How serious is it? Why was the alert raised? What evidence is available?

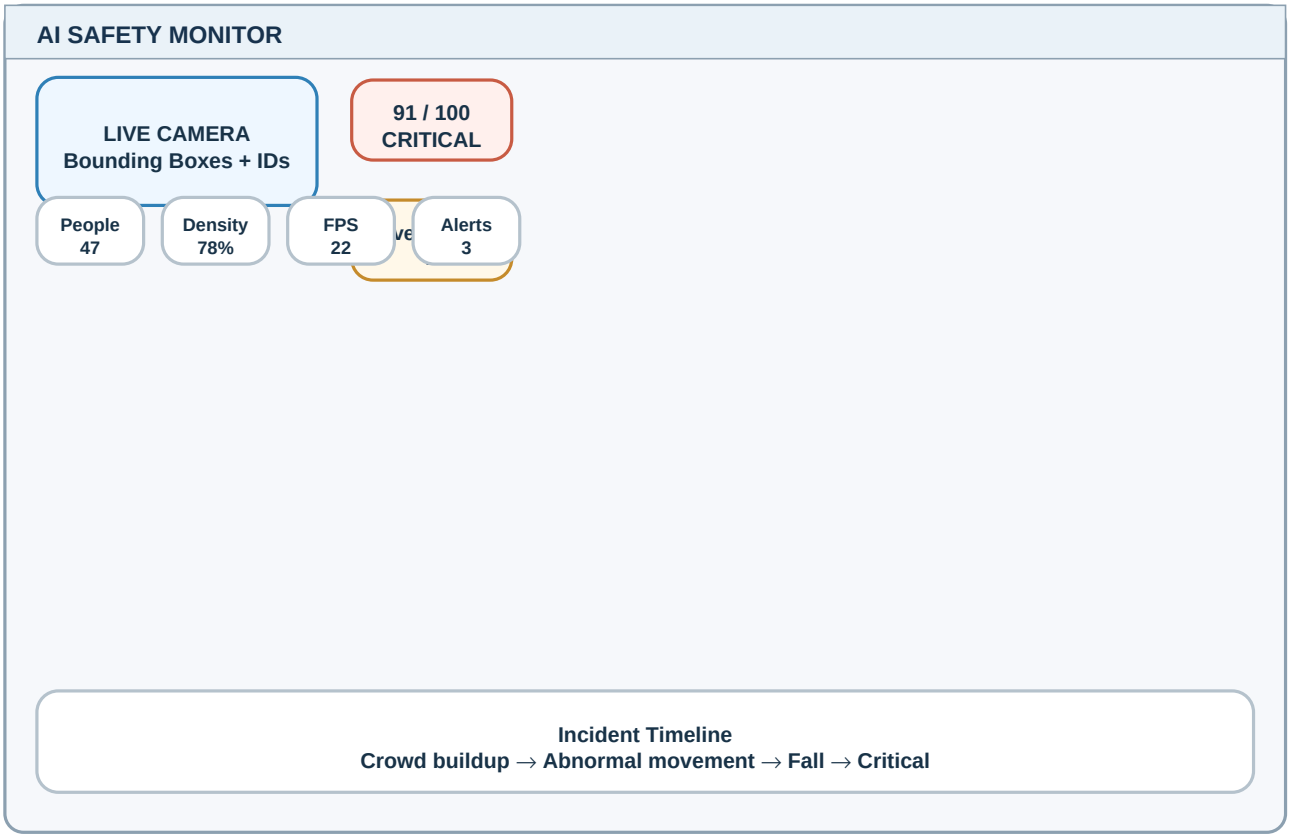


Figure 7. Conceptual dashboard layout.

Recommended dashboard sections

- **Camera grid:** camera name, online/offline state and current risk.
- **Live/near-live video:** bounding boxes, person IDs and zone overlays.
- **Active incidents:** severity, type, time and camera.
- **Risk card:** risk score and severity.
- **Incident timeline:** sequence of events that led to the alert.
- **Evidence viewer:** snapshot and short video clip.
- **Historical analytics:** incident counts, event types and trends.

Use clear severity labels such as LOW, MEDIUM, HIGH and CRITICAL. Do not rely on color alone; include text and icons.

11. Real-Time Incident Walkthrough

Imagine a college gate where the normal crowd is moderate.

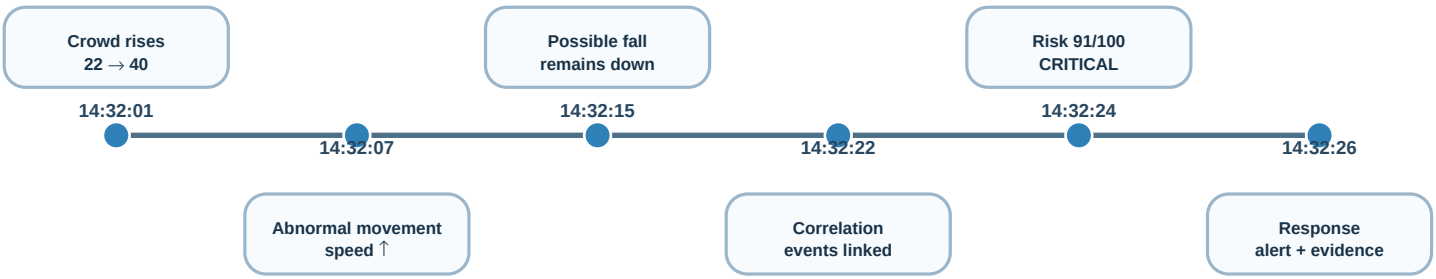


Figure 8. Example incident timeline.

Step 1 — Video input

Camera 01 continuously sends frames. The video ingestion service reads frames and passes them to the CV pipeline.

Step 2 — Detection and tracking

YOLO detects people. The tracker assigns IDs such as Person 17, Person 21 and Person 32 and maintains their trajectories.

Step 3 — Crowd buildup

The people count changes from 22 to 40 and then 55 in a short period. Density and growth-rate thresholds indicate rapid crowd buildup.

Step 4 — Abnormal movement

Several people suddenly increase speed and change direction. Movement analysis produces an abnormal-movement event.

Step 5 — Possible fall

Person 32 undergoes a rapid posture/position change and remains down for several seconds. The system creates a possible-fall event rather than immediately declaring a confirmed emergency.

Step 6 — Event correlation

The system sees crowd buildup, abnormal movement and possible fall close together in the same location. It groups them into one candidate incident.

Step 7 — Agentic reasoning

The Supervisor sends the context to the Risk Agent. The Risk Agent can query recent events and nearby cameras, calculate a risk score and return a severity.

Step 8 — Response

For a critical result such as 91/100, the Response Agent can save evidence, create an alert and request human/security attention. The exact operational action should be configurable.

Step 9 — Report and dashboard

The Report Agent creates a summary. The backend stores the incident and the dashboard receives the update through the API/WebSocket layer.

CRITICAL INCIDENT — Possible Crowd Emergency
Risk: 91/100
Why: rapid crowd buildup + abnormal movement + possible fall
Evidence: snapshot and short clip saved
Recommended action: human/security verification and monitoring of nearby cameras

12. False-Alert Reduction and Explainability

False alarms are a major practical problem in monitoring systems. A strong project should show how it avoids raising an alert from a single noisy detection.

Example: possible fall verification

```
Frame event → possible fall → observe for 5–8 seconds → person stands up → suppress high-priority alert

Frame event → possible fall → person remains down → create candidate event → correlate with nearby activity → risk assessment
```

This creates a two-stage process: detect first, verify second.

Explainable incident timeline

Time	Observed signal
14:32:01	Crowd density increased rapidly
14:32:07	Abnormal movement detected
14:32:15	Possible fall detected
14:32:22	Related events correlated
14:32:24	Risk calculated: 91/100
14:32:26	Evidence saved and alert created

The explanation should be grounded in structured system evidence. The LLM should not invent camera observations or claim certainty that the vision model did not provide.

13. Multi-Camera Extension

Once the single-camera pipeline is stable, the architecture can scale to multiple cameras.

```
Camera 01 ■■
Camera 02 ■■
Camera 03 ■■→ Event Stream → Correlation → Agentic Supervisor
Camera 04 ■■
Camera 05 ■■
```

A useful multi-camera scenario is when an event appears at one location and then continues into a nearby camera's field of view. The system can compare timestamps, zones and visual tracking features without requiring face recognition.

Example

- Camera 01 detects a person entering a restricted zone.
- The event is stored with timestamp and location.
- Camera 02 is identified as a nearby camera.
- The system checks whether related activity appears in Camera 02 within a short time window.
- The incident context is updated with supporting evidence.

Cross-camera identity should be treated carefully. For a final-year project, it is acceptable to demonstrate association using appearance features, time and spatial context without claiming perfect identity recognition.

14. Technology Stack and Recommended Tools

Layer	Recommended technology	Purpose
Programming	Python	Main AI/backend language
Video	OpenCV	Frame reading, preprocessing and video operations
Detection	YOLO	Person/object detection
Tracking	ByteTrack or BoT-SORT	Persistent IDs and trajectories
Deep learning	PyTorch	Model execution/training where required
Backend	FastAPI	REST APIs and service layer
Real-time	WebSockets	Push events to dashboard
Database	PostgreSQL	Persistent events/incidents/metadata
Agent orchestration	LangGraph	Stateful agent workflow and tool calls
LLM	Suitable API or local model	Reasoning/explanation with structured input
Frontend	React	Web dashboard
UI	Tailwind CSS	Consistent interface styling
Charts	Recharts or Plotly	Analytics
Testing	PyTest	Automated tests
API testing	Postman	Endpoint validation
Containers	Docker	Reproducible local deployment

Do not introduce every technology on day one. Start with Python + OpenCV + YOLO + tracker + FastAPI + PostgreSQL + React. Add the agent framework after structured events are working.

15. Data Structures and Module Interfaces

Integration becomes much easier if every module agrees on the same data contract before implementation.

15.1 Event contract

```
{
  "event_id": "EVT-1001",
  "camera_id": "CAM01",
  "timestamp": "2026-09-07T14:32:15",
  "event_type": "crowd_buildup",
  "confidence": 0.91,
  "people_count": 47,
  "density": 0.78,
  "metadata": {}
}
```

15.2 Incident contract

```
{
  "incident_id": "INC-104",
  "camera_id": "CAM01",
  "incident_type": "possible_crowd_emergency",
  "severity": "CRITICAL",
  "risk_score": 91,
  "related_event_ids": ["EVT-1001", "EVT-1002", "EVT-1003"],
  "explanation": "Rapid crowd buildup combined with abnormal movement and a possible fall"
}
```

15.3 Why schemas matter

- The CV module can be tested independently.
- The backend can accept events even before the final model is finished.
- The agent layer can be tested with synthetic events.
- The frontend can be developed using mock JSON.
- Integration bugs become easier to locate.

Recommended design principle: modules communicate through stable structured objects rather than sharing internal variables or model-specific code.

16. Functional and Non-Functional Requirements

16.1 Functional requirements

- The system shall accept recorded video and optionally live camera streams.
- The system shall detect people and maintain tracking IDs.
- The system shall estimate crowd count and density for configured regions.
- The system shall detect selected events such as crowd buildup, restricted-zone intrusion, loitering, abnormal movement and possible fall.
- The system shall correlate related events into incidents.
- The system shall calculate incident risk and severity.
- The system shall create alerts and store evidence metadata.
- The system shall expose incidents and camera status through APIs.
- The dashboard shall display current incidents and historical information.
- The system shall generate a concise incident report.

16.2 Non-functional requirements

- **Low latency:** event generation and dashboard updates should be near real time for the selected hardware.
- **Reliability:** temporary model/API errors should not crash the complete system.
- **Explainability:** high-risk decisions should show the evidence/signals used.
- **Scalability:** the architecture should allow multiple cameras later.
- **Security:** access to streams, evidence and administrative actions should be controlled.
- **Privacy:** avoid unnecessary personal data collection and use approved test footage.
- **Maintainability:** keep CV, backend, agents and frontend modular.

Operational safety principle

Frame predictions as possible events or risk indicators rather than guaranteed facts. High-impact responses should remain subject to human verification.

17. MSE / First Milestone — Definition of Done

The first major milestone should demonstrate a working end-to-end MVP rather than a large number of disconnected features.

MSE target pipeline

Video → YOLO → Tracking → Crowd/Zone Analysis → Event → FastAPI → PostgreSQL → React Dashboard

Minimum working capabilities

- Recorded video can be loaded and processed.
- YOLO person detection works.
- Tracking IDs are visible and stable enough for the demo.
- People count and basic crowd density are calculated.
- At least one or two safety events are generated.
- Events are sent to the backend.
- Events are stored in PostgreSQL.
- The dashboard retrieves and displays events.
- A basic risk score is visible.
- A complete end-to-end demonstration can be performed without manually moving data between modules.

Recommended priority for MSE

1. **Person detection** — foundation.
2. **Tracking** — required for temporal behavior.
3. **Crowd count/density** — highly visible in the demo.
4. **Restricted zone** — relatively easy and reliable.
5. **One temporal event** such as loitering or a basic possible-fall detector.
6. **Backend + database** — prove data persistence.
7. **Dashboard** — prove complete integration.
8. **Basic risk score** — bridge toward the final Agentic AI layer.

Advanced Agentic AI, multi-camera correlation, automated reports and richer response workflows can be added after this foundation is stable.

18. Final Project Roadmap

Phase	Main outcome
Phase 1 — Foundation	Video ingestion, YOLO detection, tracking, basic project skeleton
Phase 2 — Safety detection	Crowd, density, restricted zone, loitering, movement and fall
Phase 3 — Event system	Structured event schemas, event storage and correlation
Phase 4 — Backend	FastAPI, PostgreSQL, WebSockets and evidence metadata
Phase 5 — Dashboard	React UI, camera cards, incidents, risk and evidence
Phase 6 — Agentic AI	Supervisor, Risk Agent, Response Agent, Report Agent and tools
Phase 7 — Multi-camera	Nearby-camera context and incident correlation
Phase 8 — Research/polish	Evaluation, false-alert analysis, latency, robustness and documentation

Development order principle

Always stabilize the lower layer before depending on it. An agent cannot reason reliably about events that the CV layer does not produce consistently. The dashboard cannot be trusted until the backend schema is stable.

A good development rhythm is: implement → integrate → test → demonstrate → document.

19. Evaluation and Research Methodology

The project becomes academically stronger when it measures performance instead of only showing a visual demo.

19.1 Metrics

Area	Useful metrics
Object detection	Precision, Recall, F1, mAP
Tracking	ID consistency / tracking accuracy where ground truth is available
Event detection	Precision, Recall, F1, false-positive rate
System	FPS, detection latency, alert latency
Agentic layer	Risk classification accuracy, decision consistency, false-alert reduction

19.2 Baseline experiment

Baseline: Computer Vision event → immediate alert.
Proposed: Computer Vision event → event correlation → Agentic reasoning → risk assessment → alert.

Possible research question

“Can event correlation and agentic reasoning reduce unnecessary alerts and improve incident prioritization compared with direct event-triggered alerts?”

19.3 False-alert experiment

Create a test set containing genuine safety scenarios and harmless look-alike scenarios. Record how many alerts each approach produces and how many are actually useful.

Metric	Baseline	Proposed
Total events	X	X
False alerts	X	Y
Alert precision	X%	Y%
Average alert latency	X sec	Y sec
Correct severity classification	X%	Y%

Replace X/Y with measured values. Do not invent results for the final report.

20. Security, Privacy and Responsible Use

Video monitoring systems can process sensitive information. The project should demonstrate responsible engineering even if it is only a university prototype.

- Use public datasets, synthetic scenes or footage for which the team has permission.
- Avoid unnecessary face recognition or personal identification.
- Store only the evidence needed for the incident workflow.
- Protect API endpoints and evidence storage.
- Do not expose camera streams publicly.
- Keep an audit trail of agent decisions and tool calls.
- Clearly distinguish model predictions from confirmed facts.
- Use human verification for high-impact decisions.
- Document limitations and known failure cases.

Examples of appropriate system language

Avoid	Prefer
“Person is definitely dangerous.”	“Abnormal movement detected; human verification recommended.”
“Emergency confirmed.”	“Critical-risk candidate incident detected.”
“This person committed a crime.”	“Restricted-zone intrusion detected.”

The project's value comes from timely detection and decision support, not from making unsupported claims about people's intentions.

21. Final System Checklist

A feature is complete only when it works in the integrated system and can be demonstrated.

Component	Completion target
Video ingestion	Recorded video works reliably
YOLO	Person detections with confidence and boxes
Tracking	Persistent IDs and trajectories
Crowd analysis	Count, density and growth rate
Movement analysis	Speed/direction/abnormal movement signals
Zone analysis	Restricted polygon and intrusion event
Loitering	Duration-based event
Fall	Temporal verification and candidate event
Event schema	Stable JSON contract
Event correlator	Related events become one incident
Risk Agent	Severity and risk with explanation
Response Agent	Approved actions through tools
Report Agent	Structured incident summary
FastAPI	Events/incidents/cameras endpoints
PostgreSQL	Persistent event/incident storage
WebSocket	Near-real-time dashboard updates
React dashboard	Camera, incident, risk and evidence views
Evidence	Snapshot/clip metadata linked to incident
Evaluation	Measured metrics and baseline comparison
Documentation	Architecture, methods, results and limitations

Final architecture summary

VIDEO → COMPUTER VISION → TRACKING → BEHAVIOR ANALYSIS → EVENTS → EVENT CORRELATION → AGENTIC AI → RISK → RESPONSE → EVIDENCE/REPORT → FASTAPI → POSTGRESQL → REACT DASHBOARD

The strongest version of this project is not the one with the most AI models. It is the one where the complete pipeline works reliably, the system can explain why it raised an alert, and the team can measure how well the approach performs.

End of technical project guide